SAE S1.01-1.02 Project report

Chatbot in Java

Matys LEFRANC & Nathan REVEL

January 23, 2026

1 Introduction

For this project, we had to build a chatbot that answers general knowledge questions in French. When someone types a question, the bot searches through a database of responses and tries to find the best match.

We ended up with four Java classes. `Chatbot` is where everything starts, it handles the conversation loop with the user. `Utilitaire` is basically our toolbox with all the functions for searching and matching. `Index` is a data structure we built to store words and link them to responses. And `Thesaurus` handles synonyms so the chatbot understands that "wrote" and "author" are related. Everything is stored in text files, nothing fancy, but it works and it is easy to modify.

2 How it works

2.1 The index system

The whole point of the index is to avoid reading every response when someone asks a question. Instead of going through 1000 responses to find which ones talk about "Paris", we built something like a dictionary at the back of a book.

We have two indexes. The first one, `indexThemes`, stores content words (so not words like "the" or "is") and for each word, it keeps a list of response IDs where that word appears. If you search for "Paris", you instantly get the list of all responses mentioning Paris. No need to open each response and check.

The second one, `indexFormes`, is trickier to explain. It deals with the shape of sentences. A question like "What is the capital of France?" expects an answer that starts with "The capital of France is...". To match these patterns, we look at stop words and their positions. We store things like "what_0" meaning "what" is at position 0 in the sentence. This helps us pick responses that have the right structure.

Both indexes are sorted. Why does that matter? Because when you search in a sorted list, you do not have to check every element. You can use dichotomous search (cutting the list in half each time), which is way faster.

To fully understand how indexes work, an illustration named ‘‘`dessins-index.pdf`’’ is available in the same folder as this report.

2.2 Thesaurus

People ask the same thing in different ways. "Who invented general relativity?" and "General relativity was formulated by Albert Einstein." should give the same answer. The thesaurus handles this by mapping different words to a common form. We have a file where each line says something like "invented:formulated". When we process a question or response, we replace words with their canonical form when one exists.

It is a simple system but it makes the bot much more flexible. Without it, we would have to store separate entries for every possible phrasing.

2.3 Response selection

Finding an answer is a two step process. First, `constructionReponsesCandidates` uses the `indexThemes` to find responses that contain the keywords from the question. It gets the list of response IDs for each keyword, merges those lists together, and keeps only the IDs that appear as many times as there are keywords. That way we only keep responses that have all the words, not just one or two.

Then `selectionReponsesCandidates` filters those candidates based on form. It uses `indexFormes` to check if the shape of the response matches what the question expects. A "What is..." question should not get an answer starting with "The author of...".

There is also support for follow up questions. If you ask "What is the capital of France?" and then "And Spain?", the bot remembers the context and understands you mean the capital of Spain.

2.4 Learning mode

Users can teach the bot. When it says "I don't know", you can type "I'll teach you" and give it the answer.

But we do not just blindly add everything. First, the system checks if this response already exists by searching the index for responses with the same keywords. If it is truly new, we add it to the file and update `indexThemes`. Then we check if the question/response pair already has a form entry in `indexFormes`. If not, we create one. This double check avoids creating duplicates and keeps things organized.

3 Complexity analysis

This section compares our index based approach to a naive approach where we would check every response manually. We want to count the number of comparisons each method needs and see which one is faster.

3.1 Setting up the variables

Before we can write formulas, we need to name our quantities:

- nr = total number of responses in the database
- nmv = number of different content words across all responses
- nmr = average number of content words in one response
- nmq = number of content words in the user's question

We also need one more variable that will appear everywhere: $nrpm$, the average number of responses containing any given word. Where does it come from? Think about it this way. If we have nr responses and each has nmr words on average, then the total number of word occurrences in the database is $nr \times nmr$. These occurrences are spread across nmv different words. So on average, each word appears in:

$$nrpm = \frac{nr \times nmr}{nmv}$$

This $nrpm$ is key because it tells us how big the lists in our index are. When we look up a word, we get a list of $nrpm$ response IDs on average.

3.2 The naive approach (no index)

Without any index, here is what we would have to do. We take the question, extract its nmq keywords, then for each of the nr responses, we check if it contains all those keywords. Checking one keyword in one response means comparing it to the nmr words in that response.

So we have three nested loops:

1. Go through all nr responses
2. For each response, check each of the nmq question words
3. For each word, scan through the nmr words of the response

Three nested loops means we multiply the three quantities:

$$C_{\text{naive}} = nr \times nmq \times nmr$$

This is the total number of comparisons. With a big database, this number explodes.

3.3 Our approach with the index

Our method works in three separate steps. Each step has its own cost, and at the end we just add them up.

3.3.1 Step 1: Looking up words in the index

For each keyword in the question, we need to find it in our index to get the list of responses containing that word. The index is sorted, so we use dichotomous search. This type of search works by cutting the search space in half at each step. If you have nmv entries, you need at most $\log_2(nmv)$ comparisons to find what you want.

We do this search nmq times (once per keyword), so the total cost is:

$$\text{Step 1} = nmq \times \log_2(nmv)$$

This is usually a small number because the logarithm grows very slowly.

3.3.2 Step 2: Merging the lists

After step 1, we have nmq lists of response IDs. Each list has about $nrpm$ elements (the average number of responses per word). Now we need to merge all these lists into one big sorted list.

How does merging work? When you merge two sorted lists, you go through both simultaneously and pick the smaller element each time. The cost is roughly the sum of the two list sizes.

We merge the lists one after another. First we merge list 1 (size $nrpm$) with list 2 (size $nrpm$), which costs about $2 \times nrpm$ and gives us a list of size $2 \times nrpm$. Then we merge that result with list 3, which costs about $3 \times nrpm$. And so on until we have merged all nmq lists.

The total cost is:

$$nrpm + 2 \times nrpm + 3 \times nrpm + ... + nmq \times nrpm$$

This is the sum $1 + 2 + 3 + ... + nmq$ multiplied by $nrpm$. The formula for that sum is $\frac{nmq \times (nmq+1)}{2}$, which we can approximate as $\frac{nmq^2}{2}$.

So the cost of merging is approximately:

$$\text{Step 2} \approx \frac{nmq^2}{2} \times nrpm$$

If we replace $nrpm$ by its definition $\frac{nr \times nmr}{nmv}$, we get:

$$\text{Step 2} = \frac{nmq^2 \times nr \times nmr}{2 \times nmv}$$

3.3.3 Step 3: Finding responses with all keywords

After merging, we have one big list where each response ID appears as many times as there are matching keywords. A response containing all nmq keywords will appear nmq times in the merged list.

The `maxOccurences` function scans through this list once to find which IDs appear exactly nmq times. Since the list is sorted, equal IDs are grouped together, so we just count consecutive occurrences.

The merged list has $nmq \times nrpm$ elements (each of the nmq lists contributed $nrpm$ elements). Scanning it once costs:

$$\text{Step 3} = nmq \times nrpm = \frac{nmq \times nr \times nmr}{nmv}$$

3.3.4 Adding it all up

The total cost is just the sum of the three steps:

$$C_{\text{index}} = \underbrace{nmq \times \log_2(nmv)}_{\text{lookups}} + \underbrace{\frac{nmq^2 \times nr \times nmr}{2 \times nmv}}_{\text{merging}} + \underbrace{\frac{nmq \times nr \times nmr}{nmv}}_{\text{counting}}$$

We can factor the last two terms since they share common factors:

$$C_{\text{index}} = nmq \times \log_2(nmv) + \frac{nmq \times (nmq + 2) \times nr \times nmr}{2 \times nmv}$$

3.4 Putting numbers in

Let us see what happens with realistic values: $nr = 100,000$ responses, $nmv = 10,000$ distinct words, $nmr = 5$ words per response, $nmq = 3$ words in the question.

First, the naive approach:

$$C_{\text{naive}} = 100,000 \times 3 \times 5 = 1,500,000 \text{ comparisons}$$

Now our approach, step by step:
Step 1 (lookups): $3 \times \log_2(10,000) = 3 \times 13.29 \approx 40$
Step 2 (merging): $\frac{9 \times 100,000 \times 5}{2 \times 10,000} = \frac{4,500,000}{20,000} = 225$
Step 3 (counting): $\frac{3 \times 100,000 \times 5}{10,000} = \frac{1,500,000}{10,000} = 150$
Total: $40 + 225 + 150 = 415$ comparisons
The acceleration factor is:

$$\frac{1,500,000}{415} \approx \mathbf{3,614}$$

Our method is more than 3,600 times faster. Why such a huge difference? With the naive approach, we check all 100,000 responses even though most of them are irrelevant. With the index, we only work with responses that actually contain the words we are looking for. The index lets us ignore everything else.

4 Challenges we faced

4.1 The selectionReponsesCandidates bug

We spent a lot of time debugging `selectionReponsesCandidates`. This function filters candidates based on their form, but we kept getting errors or wrong results.

The problem was that candidate IDs and form entries were getting out of sync. When a user taught the bot something new, the response got a new ID, but sometimes the form was not computed right away. So we had IDs pointing to forms that did not exist.

We fixed it by adding bounds checking before accessing the forms array, and by making sure new responses get their forms computed immediately when they are added.

4.2 Synonyms and positions

Another tricky issue was handling synonyms in forms. Forms depend on word positions, like "what_0" for "what" at position 0. When we replace a word with its synonym, we need to keep the same position. If "who" at position 0 becomes "that" but we forget the "_0" suffix, the matching breaks.

5 AI usage

We used AI during development, but in a limited way. Mostly for debugging. When we were stuck on a function for an hour and could not see the bug, we would show it to the AI. It helped us spot issues we had missed, like off by one errors or null checks we forgot. But we always made sure to understand why the fix worked before using it. We did not want code in our project that we could not explain.

AI also helps us generate test data. Writing 50 question/answer pairs by hand is boring, so the AI helped us create about 20 more entries for our files. Same for the thesaurus, it suggested some synonym pairs we had not thought of.

We also used AI to generate the layout of this report. We first provided all the content we wanted to include, the elements of the sections and subsections, and then the AI provided us with a presentation in Latex format. We then reviewed and adapted it. The section on algorithmic complexity was calculated in advance by us, and we simply requested that it be inserted.

6 Conclusion

We managed to build a functional chatbot with fast search, synonym support, and learning capabilities. The complexity analysis shows that using an index makes our search about 3,600 times faster than a naive approach on large databases.

What we learned from this project is that choosing the right data structures really matters for performance. We also learned that keeping multiple data structures in sync is harder than it sounds. Every time we modified the index or the forms, we had to make sure everything stayed consistent.

If we had more time, we would want to add tolerance for typos, maybe a way to show how confident the bot is in its answer, and better storage than plain text files.